

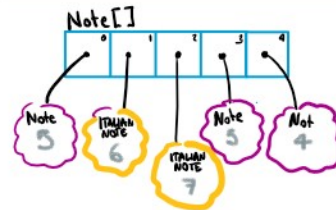
NOTE GENERALI

- LATE BINDING:** questo codice funziona poiché `ItalianNote` è un SOTTOTIPO di `Note`; il programma capisce quale usare.
- VERRIDE:** poiché sia il `Note` che `ItalianNote` dispongono di un `toString`, si dà priorità a quello del sottotipo (che sovrascrive il sovra tipo)
- PRINCIPIO DI LISKOV:** posso assegnare un sottotipo a una variabile tipo (ad esempio, `Note <-- ItalianNote`)

```
public class Main {
    public static void main(String[] args) {
        Note[] arr = new Note[5];
        int i;

        arr[0] = new Note(5);
        arr[1] = new Note(6);
        arr[2] = new Note(7);
        arr[3] = new Note(5);
        arr[4] = new Note(4);

        for (Note n : arr) {
            System.out.println(n);
        }
    }
}
```



CAMPO: E' una variabile con lo scope su tutta la classe. E' inizializzato a 0.

PRIVATE: incapsulo la variabile rendendola invisibile alle altre classi.

COSTRUTTORE: deve avere lo stesso nome della classe che costruisce.

SCRITTURA DI TEST in JUNIT

```
class Tests {
    @Test
    void semitoneIsKeptCorrect() {
        Note n = new Note(5);
        int s = n.getSemitone();
        Assertions.assertEquals(5, s);
    }

    @Test
    void testToString() {
        Note n = new Note(5);
        String s = n.toString();
        Assertions.assertEquals("Nota di semitono 5", s);
    }

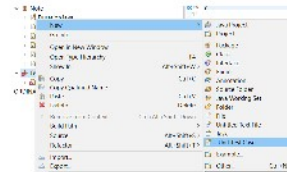
    @Test
    void testEquals() {
        Note n1 = new Note(5);
        Note n2 = new Note(5);
        Assertions.assertTrue(n1.equals(n2));
    }

    @Test
    void testNotEquals() {
        Note n1 = new Note(5);
        Note n3 = new Note(7);
        Assertions.assertFalse(n1.equals(n3));
    }
}
```

In alto deve esserci l'annotazione `@Test`. Tradizionalmente il nome deve essere `Cosadatestare.Test`

- TEST FIRST:** ho i test e cerco di scrivere un programma che soddisfi il test
- TEST AFTER:** scrivo il programma e poi scrivo i test per verificare se funziona
- NON REGRESSIONE:** scrivo i test per evitare che, modificando il programma con il tempo, il so codice peggiori.

Tests [Runner: JUnit 5] (0,063 s)
 testToString() (0,031 s)
 testNotEquals() (0,016 s)
 testEquals() (0,000 s)
 semitoneIsKeptCorrect() (0,016 s)

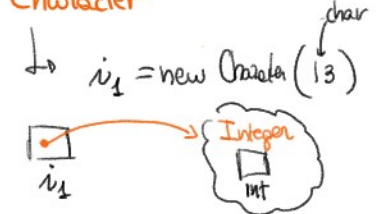


TIP WRAPPER

TIP RIFERIMENTO PER I TIP PRIMITIVI

esempio: `Character`

- + Comparable
- + late Binding
- + Metodi
- Più memoria



AUTO UNBOXING:

se assegno a un `int` un `Integer` che punta a `null`, vado in errore durante l'esecuzione!!

DICHIARAZIONE:

```
Character i1 = new Character('a'); //deprecato
Character i2 = Character.valueOf('a'); //bello
```

UTILIZZO

```
Integer i1 = Integer.valueOf(11);
Integer i2 = Integer.valueOf(12);

Integer ires = Integer.valueOf(i1.intValue()*i2.intValue());

Integer ires2 = i1*i2;
i1 == i2 funziona
```

- CLASSI -

CLASSI GENERICHE

Sono classi dichiarate con tipi generici (`<>`). Per crearle devo specificare cos'è `X` e cos'è `Y`.

CLASSI PRIVATE

DICHIARO

(STANZIO)

```
public class Pair<X,Y> {
    private final X first;
    private final Y second;

    public Pair(X first, Y second) {
        this.first = first;
        this.second = second;
    }

    public X getFirst() {
        return first;
    }

    public Y getSecond() {
        return second;
    }
}

//class MainPair {
//    public static void main(String[] args) {
//        //creiamo un'accolpa String e Song
//        //poiché il tipo è generico, devo farli notare cosa gli sto passando!
//
//        String s = "Hello";
//        Note n = new ItalianNote(5);
//        Note n2 = new ItalianNote(8);
//
//        Pair<String, Note> p1 = new Pair<String, Note>(s, n);
//        //sto specializzando la classe; è come se sostituissero String e Note in X e Y
//
//        Pair<Note, Note> p2 = new Pair<Note, Note>(n2, n);
//    }
//}

//ALTRO UTILIZZO:
public<T extends Comparable<T>> void sort(T[] arr){
    //qualunque tipo che estende Comparable
}
```



SOTTOCLASSE

```
public class ItalianNote extends Note {
    //ItalianNote EREDITA CAMPI E METODI di Note

    public final static String[] Nomi = {"DO", "DO#", "RE", "RE#", "MI", "FA", "FA#", "SOL", "SOL#", "LA", "LA#", "SI", "SI#"};

    public ItalianNote(int semitone) {
        super(semitone);
    }

    public String toString() {
        return Nomi[getSemitone()];
    }
}
```

INTERFACCIA

è un tipo che non implementa alcun metodo.

```
public class EnglishNoteWithDuration extends EnglishNote implements NoteWithDuration {
    //guarda l'interfaccia NoteWithDuration; è sensato che essa abbia il metodo getDurationOf.
    //Essendo un'interfaccia non può avere il metodo dentro, ma può averne la definizione.
    Duration getDuration();
}
```

sottointero public. Deve essere dentro le sottoclassi

CLASSI ABSTRACT

Sono classi dove ho dei metodi non ancora implementati (indicati con `abstract`). Deve esserci `abstract` anche nel nome. Non può essere istanziata in quanto incompleta.

[Deve ad avere più metodi comuni, o a per creare array.]

Sono classi dove ho dei metodi non ancora implementati (indicati con abstract). Deve esserci abstract anche nel nome. Non può essere istanziata in quanto incompleta.

ESEMPIO:

```
public abstract class Shape {
    //rappresenta una figura geometrica
    public abstract double getPerimeter();
    //??????
    public abstract double getArea();
    //??????
    public String toString() {
        return "Sono una shape di perimetro e area" + getPerimeter() + getArea();
    }
}

public class Rectangle extends Shape {
    private final double base;
    private final double height;

    public Rectangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    public double getPerimeter() {
        return (base + height) * 2;
    }

    public double getArea() {
        return base * height;
    }
}

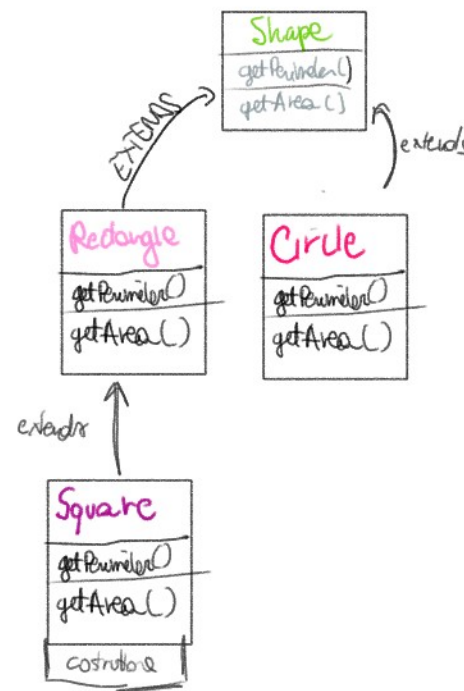
public class Circle {
    private final double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getArea() {
        return radius * radius * Math.PI;
    }

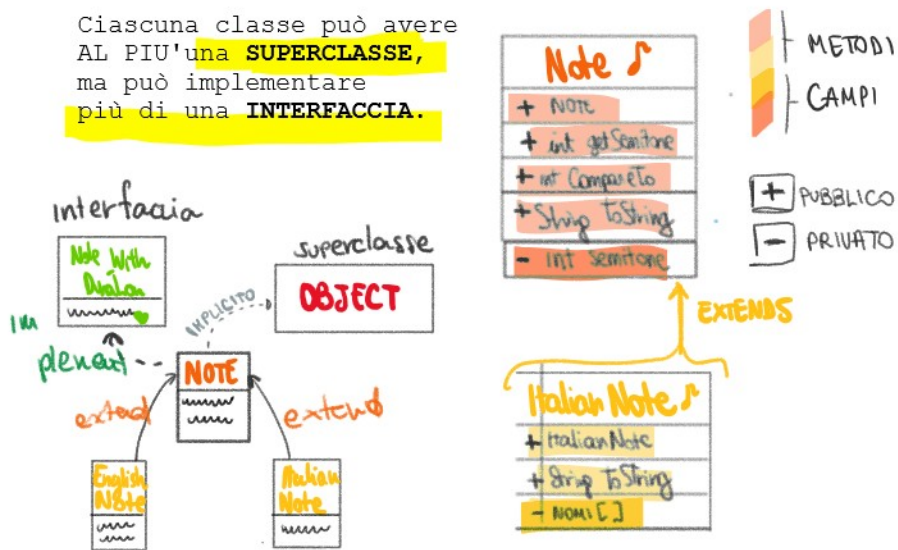
    public double getPerimeter() {
        return radius * 2 * Math.PI;
    }
}
```

[Se si ha ad avere più metodi comuni, o a poter creare array.]



RAPPRESENTAZIONE UML

Ciascuna classe può avere AL PIU' una **SUPERCLASSE**, ma può implementare più di una **INTERFACCIA**.



METODI DI OBJECT :

int compareTo	Ritorna un int che indica quale è l'oggetto minore: s1.compareTo(s2) -1 0 +1
String toString()	Stampa la classe e l'indirizzo esadecimale
boolean equals	Di default, verifica se this==other --> Va ridefinito in modo che dia true se: - I due oggetti appartengono alla stessa classe (o sovraclassa) - I due oggetti sono (logicamente) equals

```
public boolean equals(Object other) {
    return other instanceof Note && this.semitone == ((Note) other).semitone;
}
```

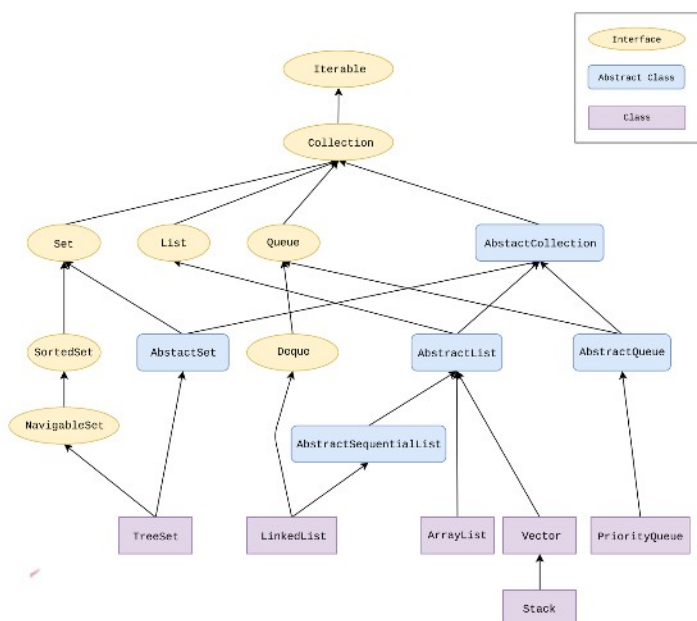
instanceOf

Ritorna true se l'oggetto è un'istanza di quella classe

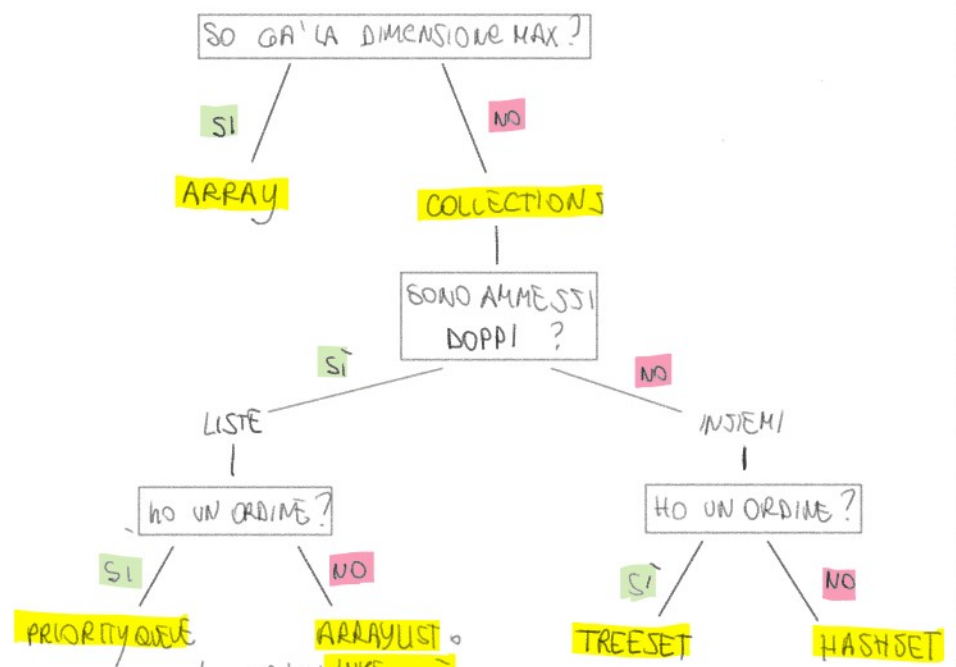
CAST

Avviso il compilatore che un oggetto è di tipo (tipo). Senza di esso non compila (non capisce che quell'oggetto ha un certo campo).

COLLEZIONI



CHEAT SHEET



Stack



① LISTE

- Arraylist

- costo lineare x ogni copia in un nuovo array
↳ $T(n)$, con $T(n)$ occasionale
- Per togliere il 1° elemento devo spostare TUTTO a dx!
↳ $T(n-1)$ + poca memoria

- Linkedlist

- spreco memoria per i puntatori + Aggiungo ed elimino in tempo costante
- per accedere all'ultimo elemento devo scorrerli tutti.

- Vectorlist - fa caghar + SINCRONIZZATO

② CODE

- Priority Queue

- Ha una priorità → gli oggetti devono essere COMPARABLE

- $\left\{ \begin{array}{l} \text{element} \\ \text{bool} \end{array} \right\} \left\{ \begin{array}{l} \text{POLL} \\ \text{REMOVE} \\ \text{offer} \\ \text{ADD} \end{array} \right.$
 - se la coda è vuota ritorna NULL
 - se la coda è vuota DA' ERRORE E BLOCCA IL PROGRAMMA
 - se la coda è piena NON FA NULLA
 - se la coda è piena DA' ERRORE E BLOCCA IL PROGRAMMA

↳ CODE BOUNDED / UNBOUNDED
= code con o senza limite di lunghezza

②/3 SORTED-SET

- TreeSet

- E' la priorityqueue dei set
- Sono tenuti in ordine crescente
- Deve usare tipi COMPARABLE

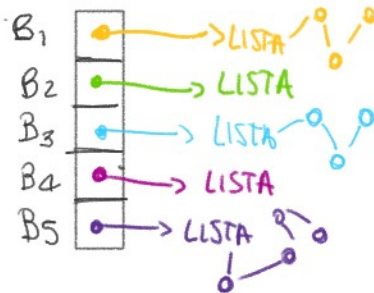
③ INSIEMI



questa patata è un insieme (a7.)

- Hashset

Per rendere più veloci le azioni, gli insiemi sono suddivisi in più BUCKETS. Sapendo in partenza in quale bucket cerare, ogni operazione diventa molto più veloce!



CONDIZIONI

- EQUALS SIMMETRICO E TRANSITIVO
- EQUALS E HASHCODE COMPATIBILI
- HASHCODE NON BANALE



Non posso modificare col for-each



I duplicati vengono rilevati con l'EQUALS -->devo ridefinirlo ove necessario



SE RIDEFINISCO EQUALS DEVO RIDEFINIRE ANCHE HASHCODE!

Se non ridefinisco hashcode succedono casini, perché l'equals e il hashcode non risultano compatibili --> mette due volte lo stesso elemento.

PRO-TIPS:

- Copio s2 e itero su quella copia; tutte le collezioni hanno un costruttore di cloni. :)

④ MAP

- HashMap

Le CHIAVI devono implementare hash e equals compatibili.

```
import java.util.HashMap;

public class MainDictionary {
    public static void main(String[] args) {
        Map<Number,String> map = new HashMap<Number, String>();
        Number n1= 123;
        map.put(n1, "centotrentaquattro");
        Number n2 = 122;
        map.put(n2,"centoventidue");
        System.out.println(map);

        String s = map.get(n1);
        System.out.println(s);

        String s1 = map.get(98);
        System.out.println(s1);
    }
}
```

```
{122=centoventidue, 123=centotrentaquattro}
centotrentaquattro
null
```